

Internship Project Proposal

Adaptive Learning Strategies for AI for Data Interpretation

Research, Implementation & Frontend Development

Project	AIDI (AI for Data Interpretation)
Duration	6 months, 15 hours/week (~360 hours total)
Primary Supervisor	Akash Amalan
Solution Architect	Kwidama Sevickson
Location	Hybrid (on-site once every 2 weeks)
Tech Stack	Python, Azure OpenAI, Databricks, React, TypeScript

Thesis Extension: This internship is designed as a prerequisite for a master's thesis. If extended to a full thesis project, academic supervision is available from **Dr. Tom Viering (TU Delft)**. The internship provides domain knowledge and identifies research gaps for deeper investigation.

1. Introduction

1.1 About AIDI

AIDI (**AI For Data Interpretation**) is a sophisticated multi-agent framework designed to automate security event extraction from enterprise onboarding documents. The system processes various document types (Word, Excel, PDF, SMIS, CRIBL configurations) through a pipeline of specialized AI agents:

- **DocumentAnalysisAgent:** Extracts security events from onboarding documents using multi-stage RAG
- **LogParserAgent:** Classifies and extracts attributes from log samples using Drain3 clustering and LLM analysis
- **DocumentOnboardingSupervisor:** Orchestrates the workflow with intelligent reclarification loops

AIDI reduces security onboarding time from 26 months to 7.2 months, with projected annual savings of €1.26M. The system is deployed on Databricks with Azure OpenAI as the LLM backbone.

1.2 The Learning Challenge

AIDI currently contains dormant learning components that are architecturally present but not activated:

BayesianAttributeLearner: Uses Beta-Bernoulli priors to estimate attribute extraction success probabilities. Designed for Thompson sampling to balance exploration vs. exploitation when prioritizing which attributes to extract.

ConfidenceAssessor: Framework for Bayesian confidence estimation with configurable thresholds (high=0.8, medium=0.5, low=0.3) and historical accuracy tracking.

DecisionEngine: Determines when to continue refinement, stop extraction, or request human intervention based on completeness thresholds and stagnation detection.

Before investing significant engineering effort to fully activate and integrate these components, we need to validate that the chosen approach is appropriate for our use case and understand its limitations.

1.3 Internship Objective

This internship combines **research** (investigating optimal learning strategies), **backend development** (activating and improving learning components), and **frontend development** (migrating UI to React). The primary deliverable is a comprehensive **survey and implementation plan** that guides future development of AIDI's learning capabilities.

2. Research Component

The research component investigates how AIDI should learn from extraction feedback. The intern will address the following pre-defined research questions through literature review, experimentation, and analysis.

2.1 Research Questions

RQ1: Baseline Validation

Question: Does the current Bayesian approach (Beta-Bernoulli priors with Thompson sampling) outperform simple baselines for attribute prioritization?

Baselines to compare:

- Random ordering: Shuffle attributes randomly
- Fixed ordering: Mandatory attributes first, then optional (current default)
- Frequency-based: Order by historical success rate without uncertainty modeling

Success metric: Cumulative regret, iterations to reach 90% completeness, sample efficiency

RQ2: Context Sensitivity

Question: Can incorporating contextual information (document type, event category, vendor) significantly improve learning performance compared to context-free Thompson sampling?

Approaches to evaluate:

- Context-free: Current approach with global priors per attribute
- Stratified: Separate priors per document type or event category
- Contextual bandits: LinUCB or similar with feature vectors

Success metric: Relative improvement over context-free baseline across heterogeneous documents

RQ3: Credit Assignment

Question: When extraction fails in the multi-agent pipeline, how should credit (or blame) be assigned to individual agents to enable effective learning?

Approaches to evaluate:

- End-to-end: Single reward signal for entire pipeline (simple but noisy)
- Cascading confidence: Each agent outputs confidence; credit = product of confidences
- Counterfactual: What would have happened with different agent outputs?

Success metric: Learning speed and final performance compared to end-to-end baseline

RQ4: Sample Efficiency

Question: How many human feedback examples are required before the learning system shows statistically significant improvement over the non-learning baseline?

Analysis approach:

- Learning curves: Plot performance vs. number of feedback examples
- Statistical testing: At what N does performance significantly exceed baseline ($p < 0.05$)?
- Practical threshold: Is N achievable within realistic operational constraints?

Success metric: Number of examples to achieve 10% improvement over baseline

2.2 Research Methodology

Literature Review

Survey existing work on:

- Multi-armed bandits and Thompson sampling in production systems
- Contextual bandits for personalization and recommendation
- Credit assignment in multi-agent reinforcement learning
- Active learning and sample-efficient learning from human feedback
- Learning for information extraction and NLP tasks

Deliverable: Literature review document (8-12 pages) with taxonomy of approaches and applicability analysis for AIDI.

Experimentation

Build a simulation framework to test learning approaches without expensive LLM calls:

- Synthetic extraction tasks with configurable success probabilities
- Attribute correlation structures (independent, clustered, hierarchical)
- Document type distributions matching real AIDI usage patterns
- Noise injection to simulate real-world feedback quality

Validate promising approaches with small-scale real data experiments using held-out AIDI extractions.

3. Backend Development

The intern will work hands-on with AIDI's learning infrastructure, activating dormant components and implementing improvements based on research findings.

3.1 Codebase Familiarization

Key files and modules to understand:

Component	Location
Bayesian Learner	aidi/agents/supervisor/components/learning/bayesian_learner.py
Confidence Assessor	aidi/agents/supervisor/components/quality/confidence_assessor.py
Decision Engine	aidi/agents/supervisor/components/decision/engine.py
Workflow Manager	aidi/agents/supervisor/agent/workflow.py
Metrics Calculator	aidi/agents/supervisor/components/quality/metrics_calculator.py
Supervisor Tracker	aidi/agents/supervisor/components/tracking/tracker.py

3.2 Tasks

Activate Learning Components

- Remove DORMANT flags from BayesianAttributeLearner and ConfidenceAssessor
- Implement full Thompson sampling logic for attribute prioritization
- Wire up feedback ingestion: extraction outcomes update priors
- Persist learned parameters to Delta Lake for continuity across sessions

Testing and Validation

- Unit tests for prior updates, Thompson sampling, and prioritization logic
- Integration tests for end-to-end learning loop with simulated feedback
- Property-based tests to verify Bayesian update mathematics
- Performance tests to ensure learning overhead < 5% of extraction time

Improvements Based on Research

- If RQ2 shows benefit: Implement context-aware priors or contextual bandit extension
- If RQ3 shows benefit: Implement cascading confidence for credit assignment
- Refactor WorkflowManager to support pluggable learning strategies (Strategy pattern)

3.3 Code Quality Standards

- Type hints on all new functions (Python 3.10+ style)
- Docstrings following Google style guide
- No hardcoded values; all thresholds configurable via YAML
- Builder pattern for complex objects (consistent with existing codebase)
- Comprehensive logging at appropriate levels (DEBUG, INFO, WARNING)
- Target: 80%+ test coverage for learning components

4. Frontend Development

AIDI's frontend is currently built with Streamlit. The intern will help migrate to a React-based architecture for better performance, maintainability, and user experience.

4.1 Current State (Streamlit)

The existing frontend includes:

- Document Analysis tab: File upload, extraction trigger, results display with expandable event cards
- Log Parser tab: Schema input, log processing, query interface
- Supervisor tab: End-to-end pipeline orchestration
- Analytics tab: Execution timeline, onboarding details, error debugging
- All Onboardings tab: Overview of all projects and their status

4.2 Target Architecture (React)

Technology stack:

- React 18 with TypeScript for type safety
- Tailwind CSS for styling (matches Cribl design patterns)
- React Query for server state management and caching
- Recharts or Visx for data visualizations
- FastAPI backend wrapper for existing Python services

4.3 Migration Tasks

Core Components

- File upload with drag-and-drop and progress indication
- Results display with expandable cards, attribute tables, and status badges
- Analytics dashboard with charts and filtering
- Sidebar navigation with connection status indicators

New Components (Support Learning Research)

- Feedback collection UI: Inline correction buttons on each extracted attribute
- Correction modal: Original value, corrected value, optional reason
- Batch review mode: Efficiently review multiple extractions
- Learning insights dashboard: Prior evolution charts, confidence calibration plots

Quality Requirements

- Responsive design for desktop and tablet
- Accessible (WCAG 2.1 AA compliance)
- Unit tests for complex components (React Testing Library)
- Consistent with existing Cribl branding guidelines

5. Timeline

The internship spans 6 months at 15 hours per week (~360 hours total). The timeline balances research, backend, and frontend work with increasing integration toward the end.

Month	Focus Areas	Deliverables
1	Onboarding: Codebase familiarization, understand learning components, set up dev environment, initial literature search	Dev environment ready, codebase overview document
2	Research: Deep literature review; Backend: Activate BayesianAttributeLearner with basic tests; Frontend: React project setup	Literature review draft, activated learner with tests, React scaffold
3	Research: Build simulation framework, run RQ1 experiments; Backend: Integrate learning into pipeline; Frontend: Migrate Document Analysis tab	Simulation framework, RQ1 results, integrated learning, React Document Analysis
4	Research: RQ2-RQ3 experiments; Backend: Implement improvements based on findings; Frontend: Migrate remaining tabs	RQ2-RQ3 results, improved learning components, React tabs
5	Research: RQ4 analysis, write survey; Backend: Polish and document; Frontend: Feedback UI and learning dashboard	Survey draft, documented backend, feedback UI
6	Finalize survey and implementation plan, integration testing, final presentation preparation, handover documentation	Final survey, implementation plan, presentation, handover docs

Bi-weekly check-ins with supervisors (Akash Amalan, Kwidama Sevickson). Monthly progress reviews with broader team.

6. Deliverables

6.1 Survey Document

A comprehensive survey (15-25 pages) covering:

- Literature review: Taxonomy of learning approaches for document intelligence
- AIDI context: Current architecture, constraints, and requirements
- Experimental results: Findings for RQ1 through RQ4 with statistical analysis
- Comparative analysis: Strengths and weaknesses of evaluated approaches
- Recommendations: Clear guidance on which approach to adopt and why

6.2 Implementation Plan

A detailed implementation roadmap (5-10 pages) including:

- Recommended architecture for AIDI's learning system
- Phased implementation plan with effort estimates
- API specifications for learning components
- Integration points with existing codebase
- Testing strategy and success criteria

- Risk assessment and mitigation strategies
- Future research directions (potential thesis topics)

6.3 Code Deliverables

- Activated BayesianAttributeLearner with full Thompson sampling
- Activated ConfidenceAssessor with calibration support
- Simulation framework for testing learning approaches
- Test suite with 80%+ coverage for learning components
- React frontend with core tabs migrated and feedback UI

6.4 Documentation and Presentation

- Updated README and inline documentation for learning components
- React component documentation and architecture decision records
- Final presentation (45-60 minutes) to the team
- Handover document with outstanding work and recommendations

7. Thesis Extension

This internship is designed as a prerequisite for a Master's thesis at TU Delft. The internship provides domain knowledge, identifies research gaps, and produces preliminary results that can be extended into a full thesis.

7.1 Thesis Supervision

Academic Supervisor: Dr. Tom Viering (TU Delft)

Industry Supervisors: Akash Amalan (Primary), Kwidama Sevickson (Solution Architect)

7.2 Potential Thesis Topics

Based on internship findings, the thesis could explore:

- Contextual bandits for adaptive document intelligence
- Credit assignment in multi-agent LLM systems
- Sample-efficient learning from human feedback in enterprise AI
- Transfer learning across document types and domains
- Online learning with non-stationary reward distributions

7.3 Thesis Timeline (If Extended)

Additional 6-9 months for thesis work

- Months 1-2: Deep dive into selected research direction
- Months 3-5: Implementation and extensive experimentation

- Months 6-7: Analysis and thesis writing
- Month 8-9: Review, revision, and defense

The thesis would aim for publishable results at venues such as EMNLP, ACL, or ICML workshops.

8. Skills and Growth

8.1 What You Will Learn

- Multi-agent AI systems and LLM orchestration (LangChain, LangGraph)
- Bayesian methods, multi-armed bandits, and reinforcement learning
- Production ML infrastructure: MLflow, Databricks, Delta Lake
- Modern frontend development: React, TypeScript, Tailwind
- Research methodology: Literature review, experiment design, technical writing
- Enterprise software development practices and code quality standards

8.2 Prerequisites

- Python proficiency (required)
- Basic machine learning knowledge (required)
- React/TypeScript experience (helpful but not required)
- Interest in AI systems and research (required)
- Self-directed work style with periodic guidance (required)